CP Wing - C++ Standard Template Library

Containers · Ordered vs Unordered · Nesting · Binary Search

Essential STL Containers & Algorithms for Competitive Programming

Objectives

- **Master the core Standard Template Library (STL) tools for Competitive Programming:**
  - Arrays and Vectors — contiguous memory, dynamic resizing, iterators
  - Maps and Sets — key-value lookup, set uniqueness, and why we favour ordered containers
  - Stacks and Queues — LIFO / FIFO adapters, use cases
  - Nesting & Binary Search — multi-dimensional containers, CCTV intuition, lower/upper_bound

C-Style Arrays

- A C-style array is simply a block of contiguous memory.
- First element is always index 0.
- Note the < in the for loop. If you put <= you will certainly have trouble!
  - *If you get a "runtime error" from the judge, check for that.*
- Access an element by index with brackets: O(1) time. Very fast!
  - *If you have to "look for" an element, it's O(n) time. Use with caution.*

```
int arr[100];
int i;

for (i = 0; i < 100; ++i) {
    arr[i] = i * 10;
}
```

C-Style Arrays, initialization

- You can initialize arrays inline in C if you need to.
- Note that uninitialized items are undefined!
  - You don't have to initialize right away, but you have to before you use it!

```
int foo[10] = {8, 6, 7, 5, 3, 0, 9};

char suits[4] = "SHCD";  // Spades, Hearts, Clubs, Diamonds
```

Vectors

- Vectors in C++ are awesome. Use them unless you have good reason not to.
  - They can grow dynamically! No need to determine the proper size in advance.
  - Many iterators to provide traversals.
  - Reasonable default initialization.
  - Use push_back to insert an element at the end.
    - *Inserting at the beginning is slow! Don't do it!*

```
vector<int> foo;

for (int i = 0; i < N; ++i) {
    cin >> data;
    foo.push_back(data);
}
```

Vector initializations

- The constructor can initialize the vector for you.
  - One argument n: n instances of the default value.
  - Two arguments n and x: n copies of x.

```
vector<int> foo(100);          // 100 integers initialized to 0

vector<int> foo(500, 123);     // 500 integers initialized to 123
```

- You can use foo.reserve(1000) to pre-allocate space without resizing.

Looping

C Style

```
int sum = 0;
for (i = 0; i < foo.size(); ++i)
    sum += foo[i];
```

Iterator Style

```
int sum = 0;
for (auto i = foo.begin(); i != foo.end(); ++i)
    sum += *i;
```

Reverse Iterator Style

```
int sum = 0;
for (auto i = foo.rbegin(); i != foo.rend(); ++i)
    sum += *i;
```

Vector methods

Function	Purpose
v.size()	Number of elements
v.empty()	Whether it contains zero elements
v.push_back(x)	Append a copy of x
v.emplace_back(args...)	Construct an element at the end
v.pop_back()	Remove the last element
v.front()	Access the first element

Function	Purpose
v.back()	Access the last element
v.at(i)	Bounds-checked access
v[i]	Unchecked indexed access
v.clear()	Remove all elements
v.resize(n)	Change the number of elements
v.reserve(n)	Request capacity for at least n elements
v.capacity()	Current allocated capacity

Pairs

- It is often convenient to define tuples / pairs as well.

```
pair<int, int> coord;

coord.first  = 10;
coord.second = 999;
```

Sets

- Use std::set when you need to maintain unique elements in sorted order.
  - Under the hood, std::set is a balanced binary search tree (Red-Black tree).
  - Guaranteed O(log N) lookup, insertion, and deletion — always predictable.
  - Supports lower_bound() and upper_bound() for binary search queries in O(log N).

```cpp
// #include <bits/stdc++.h>
using namespace std;

int main() {
    set<string> people;
    people.insert("Jenni");
    people.insert("emergency");
    people.insert("Empire");

    if (people.count("Jenni")) {
        cout << "Found Jenni!" << endl;
    }
}
```

Motivation for a map!

- Arrays are fun, but what's with all the integers?
  - *Maps allow you to look up a value by supplying any arbitrary key.*
  - E.g., name / phone number, word / definition, coordinate / visit count
- Maps can find any associated value quickly.
- Sets are like maps, but we only store unique keys (no values).
- **These, with arrays, are easily the most fundamental data structures you can master.**

Operations

- **We will explore these core map operations for C++:**
  - *Declaring or Creating the map*
  - *Insert a key-value pair into the map*
  - *Lookup a value given a key*
  - *Check if a key is in the map*
  - *Query the size*
  - *Iterate over the keys or the values*
  - *Remove a key from the map*

Creating and Inserting

- To create a map in C++, use the std::map container class.
  - Provide the key and value types as templates: map<KeyType, ValueType>.
- Insertion has two primary forms:
  - "array like" insertion via brackets [ ]
  - "pair" insertion using .insert({key, value})

```
// #include <bits/stdc++.h>
using namespace std;

int main() {
    map<string, int> phonebook;
    phonebook["Jenni"] = 8675309;
    phonebook["emergency"] = 911;
    phonebook.insert({"Empire", 5882300});
}
```

In-line initialization

- You can also initialize a map at compile-time using an initializer list.
  - Useful for pre-baked lookup tables or mapping constant tokens in CP.

```
map<string, int> phonebook = {
    {"Jenni", 8675390},
    {"emergency", 911},
    {"Empire", 5882300}
};
```

Lookup

- To lookup a specific value, you have multiple options:
- Use array syntax [ ] if you know the key is there.
  - **WARNING: Bracket lookup creates the key with default value if it doesn't already exist!**

```
cout << phonebook["Jenni"] << " and "
     << phonebook["H"] << endl;
```

- *Output: 8675309 and 0 (phonebook["H"] was inserted automatically as 0!)*

Finding Keys

- To safely check if a key exists without creating it:
  - **Use .contains("key") in C++20, or .count("key") > 0 in C++11/14/17.**

```
if (phonebook.contains("H")) {
    cout << "H is " << phonebook["H"] << endl;
}
```

- *Finding by value is not supported by standard map lookup — that requires a full scan O(N).*

Size

- To get the number of pairs in the map, use .size() — O(1) time.
- To check if it is empty, use .empty() — faster and cleaner than size() == 0.

```
if (phonebook.empty()) {
    cout << "We don't know anyone." << endl;
} else {
    cout << "There are " << phonebook.size() << " entries." << endl;
}
```

Iteration

- To loop over all entries, use range-based for loops or iterators.
  - std::map is strictly sorted by key (Red-Black tree in-order traversal).
- Each element is a std::pair<const Key, Value> — access with it->first and it->second.
  - In modern C++17, use structured bindings: for (auto [k, v] : phonebook).

```
for (auto it = phonebook.begin(); it != phonebook.end(); ++it) {
    cout << it->first << " has phone number "
         << it->second << endl;
}

// Modern C++17 structured binding alternative:
for (auto [name, number] : phonebook) { ... }
```

Ordered vs. Unordered: Why We Favour set & map in CP

- **std::set & std::map (Ordered — Balanced Red-Black Trees):**
  - Guaranteed O(log N) operations in ALL cases (best, average, and worst).
  - Elements are always sorted; supports lower_bound() / upper_bound().

- **std::unordered_set & std::unordered_map (Hash Tables):**
  - **Average O(1) operations, but WORST-CASE O(N) when hash collisions occur!**

- **Why We Strictly Favour std::set & std::map in Competitive Programming:**
  - The Codeforces Hacking Risk: On Codeforces, other contestants can view your solution and craft custom test inputs during the hacking phase.
  - **Anti-Hash Tests: Adversarial inputs can be easily designed to force massive hash collisions in the built-in std::hash, degrading lookups to O(N).**
  - **This blows total time up from O(N) to $O(N^2)$ → your solution gets Hacked / TLE!**
  - **std::set & std::map are immune to hash attacks. For $N = 2 \cdot 10^5$, $\log_2(N) \approx 18$ operations — fast, reliable, and 100% hack-proof!**

Stacks

- LIFO data structure (Last-In, First-Out).
- Stacks have three primary operations, all O(1) time:
  - push(x) — add x to the top of the stack: O(1)
  - pop() — remove the top element from the stack: O(1)
  - top() — return the top element without removing it: O(1)

```
stack<int> s;
s.push(10);  s.push(20);  s.push(30);

while (!s.empty()) {
    cout << s.top() << endl;
    s.pop();
}
// outputs: 30, 20, 10  (reverse of insertion order)
```

Stack Use Case

- Classic contest problem: Matching parentheses & brackets.

```
stack<char> s;
char c;
while (cin >> c) {
    if (c == '(') s.push(c);
    else if (c == ')') {
        if (s.empty()) { isValid = false; break; }
        s.pop();
    }
}
```

- Also essential in Depth First Search (DFS), monotonic stack problems, and cycle detection.
  - Note: std::vector provides push_back() and back(), serving as a lightweight stack!

Queues

- FIFO data structure (First-In, First-Out).
- Queues have three primary operations, all O(1) time:
  - push(x) — add x to the back of the queue (enqueue): O(1)
  - pop() — remove the front element from the queue (dequeue): O(1)
  - front() — return the front element without removing it: O(1)

```
queue<int> q;
q.push(10);  q.push(20);  q.push(30);

while (!q.empty()) {
    cout << q.front() << endl;
    q.pop();
}
// outputs: 10, 20, 30  (preserves arrival order)
```

Queue Use Cases

- **You will see queues everywhere in competitive programming!**
- Many fundamental graph algorithms rely on queues:
  - Breadth First Search (BFS) — finding shortest paths in unweighted graphs
  - Topological sorting (Kahn's algorithm using in-degrees)
  - Multi-source BFS

- *Vectors are not a good replacement for queues (popping from the front is O(N)!).*

Nesting in STL Containers

- Real contest problems often require multi-dimensional or grouped data.
- **STL containers can be freely nested inside one another without writing custom classes:**
  - vector<vector<int>>: 2D Dynamic Grid
  - map<string, vector<int>>: Inverted Index (groups list of occurrences/indices by key)
  - set<pair<int, int>>: Unique 2D coordinates (x, y)

```
// 1. 2D dynamic grid: N rows x M cols initialized to 0
vector<vector<int>> grid(n, vector<int>(m, 0));
grid[r][c] = 42;

// 2. Map from string -> list of occurrences
map<string, vector<int>> occurrences;
occurrences["apple"].push_back(12);
```

Binary Search: The CCTV Bike Mystery

- **The Scenario: A bike was parked at 12:00 PM and found missing at 6:00 PM (360 minutes).**
- When was it stolen? We have 360 minutes of CCTV footage to search:
  - *Linear Search (O(N)): Watch every single minute from 0 to 360 → 360 checks (too slow!).*
  - **Binary Search (O(log N)): Check the midpoint each step to halve the search space!**
    - Minute 180 (3:00 PM): Bike still there? YES → Stolen in second half [181...360]!
    - Minute 270 (4:30 PM): Bike still there? NO → Stolen between [181...270]!
    - Minute 225 (3:45 PM): Bike still there? YES → Keep halving remaining window...
- **The Core Condition: Monotonicity [YES, YES, YES, ..., YES, NO, NO, ..., NO]**
  - **Binary search finds the exact minute of the theft in just $\lceil \log_2(360) \rceil = 9$ checks!**

```cpp
int lo = 0; // the bike was there at minute 0
int hi = 360; // the bike was missing at minute 360
while(hi - lo > 1) {
    int mid = lo + (hi - lo)/2;

    if (/* bike present at minute mid */) lo = mid;
    else hi = mid;
}
cout << lo << endl;
```

std::lower_bound & std::upper_bound

- **GOLDEN RULE: The array / vector MUST BE SORTED beforehand! (Otherwise undefined).**
- Both perform binary search in O(log N) time on sorted sequences:
  - **lower_bound(begin, end, x): Points to the FIRST element that is ≥ x (not less than x).**
  - **upper_bound(begin, end, x): Points to the FIRST element that is > x (strictly greater).**
- **Visual Example with duplicates: a = [10, 20, 20, 20, 30, 40]**
  - Query x = 20:
    - lower_bound points to the FIRST 20 (index 1)
    - upper_bound points to 30 (index 4 — the first value strictly > 20)
- **Contest Shortcut: Count of x in sorted array = upper_bound - lower_bound in O(log N)!**

Iterator Arithmetic: lower_bound - a.begin()

- **The Pitfall: lower_bound returns an ITERATOR (pointer-like), NOT an integer index!**
- To get the 0-based integer index, subtract the beginning iterator:
  - **For vector: int idx = lower_bound(a.begin(), a.end(), x) - a.begin();**
  - **For raw array: int idx = lower_bound(arr, arr + n, x) - arr;**
- **What Does idx Represent? Three Critical Cases to Know:**
  - 1. Element Exists: idx is the index of its first occurrence (a[idx] == x).
  - 2. Element Missing: idx is the index where x SHOULD be inserted to keep array sorted.
  - **3. All elements < x: Returns a.end(), so idx == a.size() (out of bounds!).**

```
vector<int> a = {10, 20, 20, 30, 40};
int idx = lower_bound(a.begin(), a.end(), 20) - a.begin(); // idx = 1

// Always guard against out-of-bounds when verifying element presence:
if (idx < a.size() && a[idx] == 20) {
    cout << "Found at index " << idx << "\n";
}
```